
Intelligent Traffic Signal Optimization Using Reinforcement Learning in SUMO

Introduction

- Traffic congestion is one of the biggest challenges in urban transportation systems.
- Traditional traffic signals rely on predefined timing plans, which cannot adjust to sudden changes such as peak-hour surges, accidents, or uneven vehicle arrival patterns.
- With the growth of smart cities, there is a need for adaptive and intelligent traffic control that responds to real-time traffic conditions.
- Reinforcement Learning (RL) offers a data-driven, self-learning approach for traffic signal optimization.
- In this project, we use SUMO and TraCI to simulate a real-time intersection and train RL agents (Q-Learning and DQN) to minimize congestion and improve traffic flow efficiency.

Problem Statement

Current Issues:

- Urban road intersections suffer from congestion and long waiting times.
- Traditional fixed-time signals cannot adapt to dynamic traffic volume.
- Manual control is inefficient, inconsistent, and not scalable.
- Need for a learning-based adaptive traffic signal capable of real-time optimization.
- Modern traffic requires intelligent systems that minimize delay and improve throughput.

Objectives

- Build a unified SUMO-based simulation for traffic signal control.
- Compare Static, Tabular Q-Learning, and DQN models.
- Use E1 lane area detectors for realistic queue measurement.
- Analyze performance using real-world ITS metrics:
 - Total delay
 - Queue length
 - Travel time
 - Throughput
- Demonstrate traffic improvements through RL.



Literature Review

1. Traditional Optimization Approaches

- Websters (1958), SCOOT (1991), SCATS (1996)
- Use fixed or semi-adaptive timing
- Do NOT learn from environment → perform poorly under non-linear traffic

2. Reinforcement Learning Approaches

Papers show RL improves mobility significantly:

- Wei et al., “PressLight,” KDD 2019 — Max-pressure RL improves flow coordination
- Zheng et al., “IntelliLight,” KDD 2019 — Deep RL reduces delay by 25%
- Aslani et al., ESWA 2022 — RL outperforms classical control on multiple intersections





Literature Review:

3. Deep RL for Traffic Signals

- van der Pol, NeurIPS 2016 — DQN reduces congestion patterns
- Lin & Zheng, IEEE TVT 2022 — Dueling-DQN stabilizes queues

Research Gap Observed:

- Very few studies compare tabular RL vs deep RL in the same SUMO environment
- Limited usage of lane-level data (detector-based)
- Scarcity of real-time online learning experiments

Methodology

SUMO Simulation Setup

- 4-way intersection
- Vehicles flowing from multiple directions
- Lane-level detectors placed at strategic positions to capture:
 1. Queue length
 2. Vehicle counts
- Python controls the traffic signal at every timestep
- SUMO GUI used for real-time visualization

Methodology

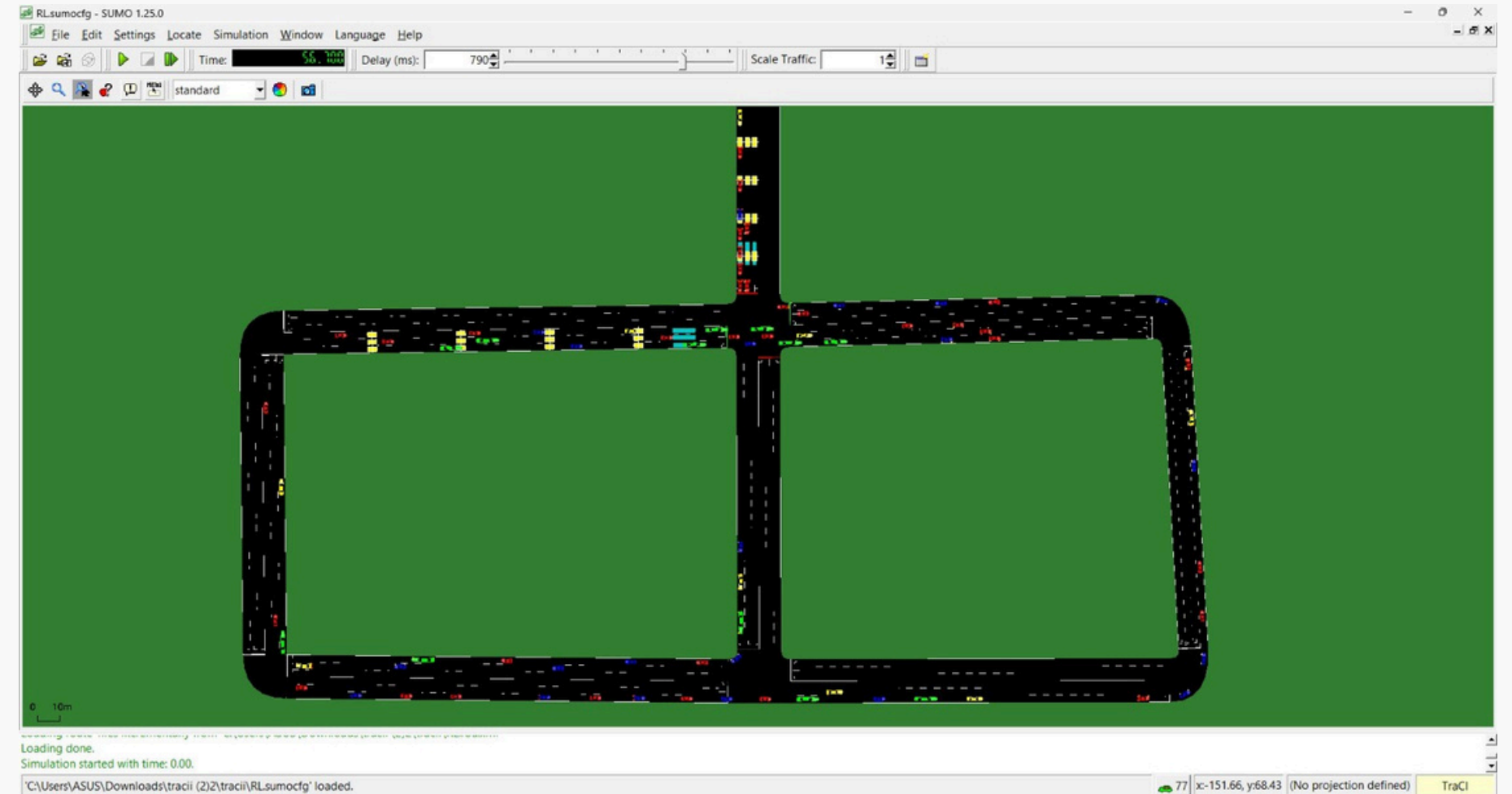
TraCI & Real-Time Control

TraCI (Traffic Control Interface) enables:

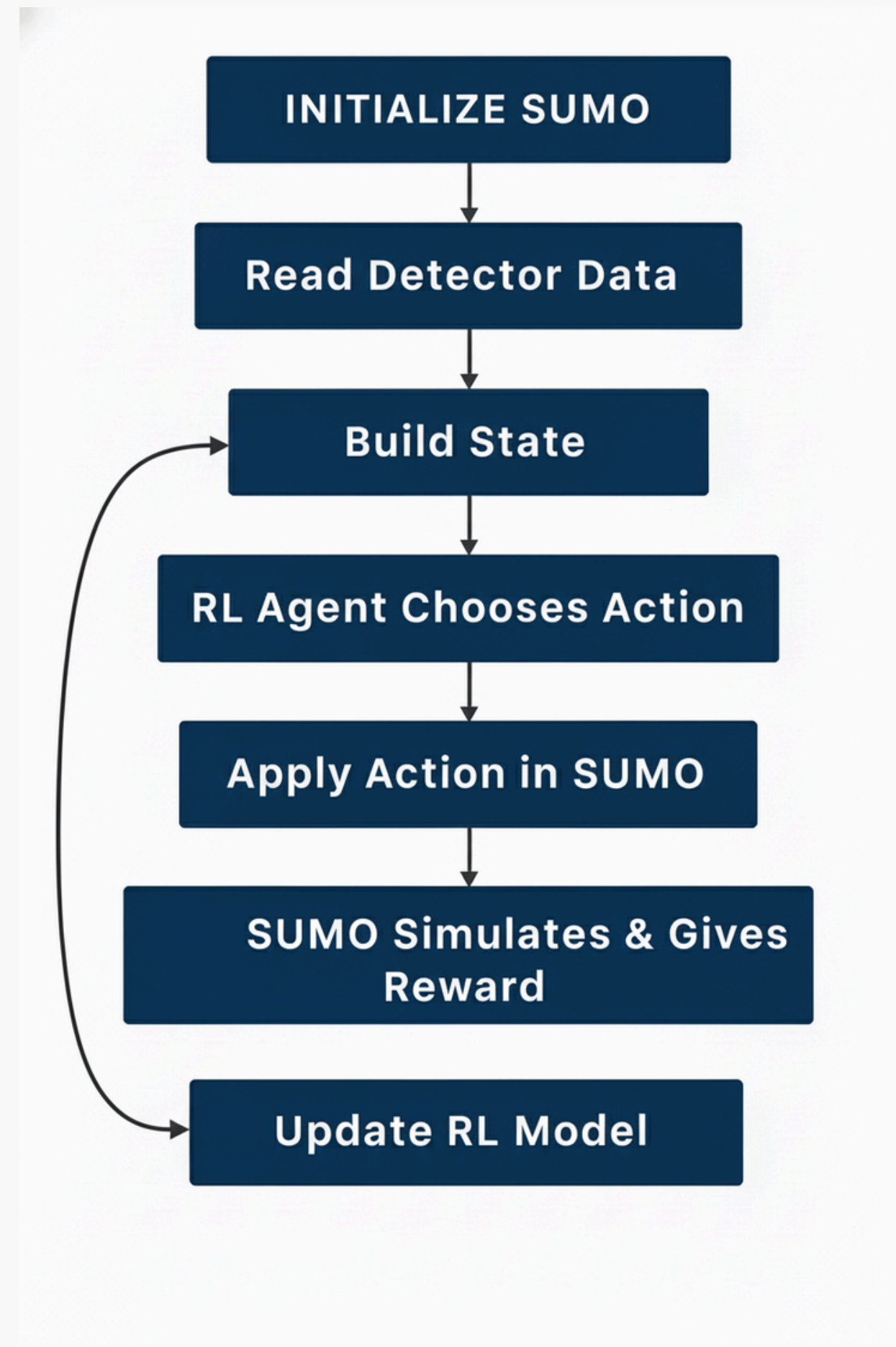
- Reading detector values
- Getting current signal phase
- Switching phases dynamically
- Step-by-step communication with SUMO

The RL agent:

- Observes traffic state
- Chooses an action (keep/switch signal)
- Receives reward
- Learns to reduce congestion



Flow Chart-



Reinforcement Learning (RL) for Signal Control

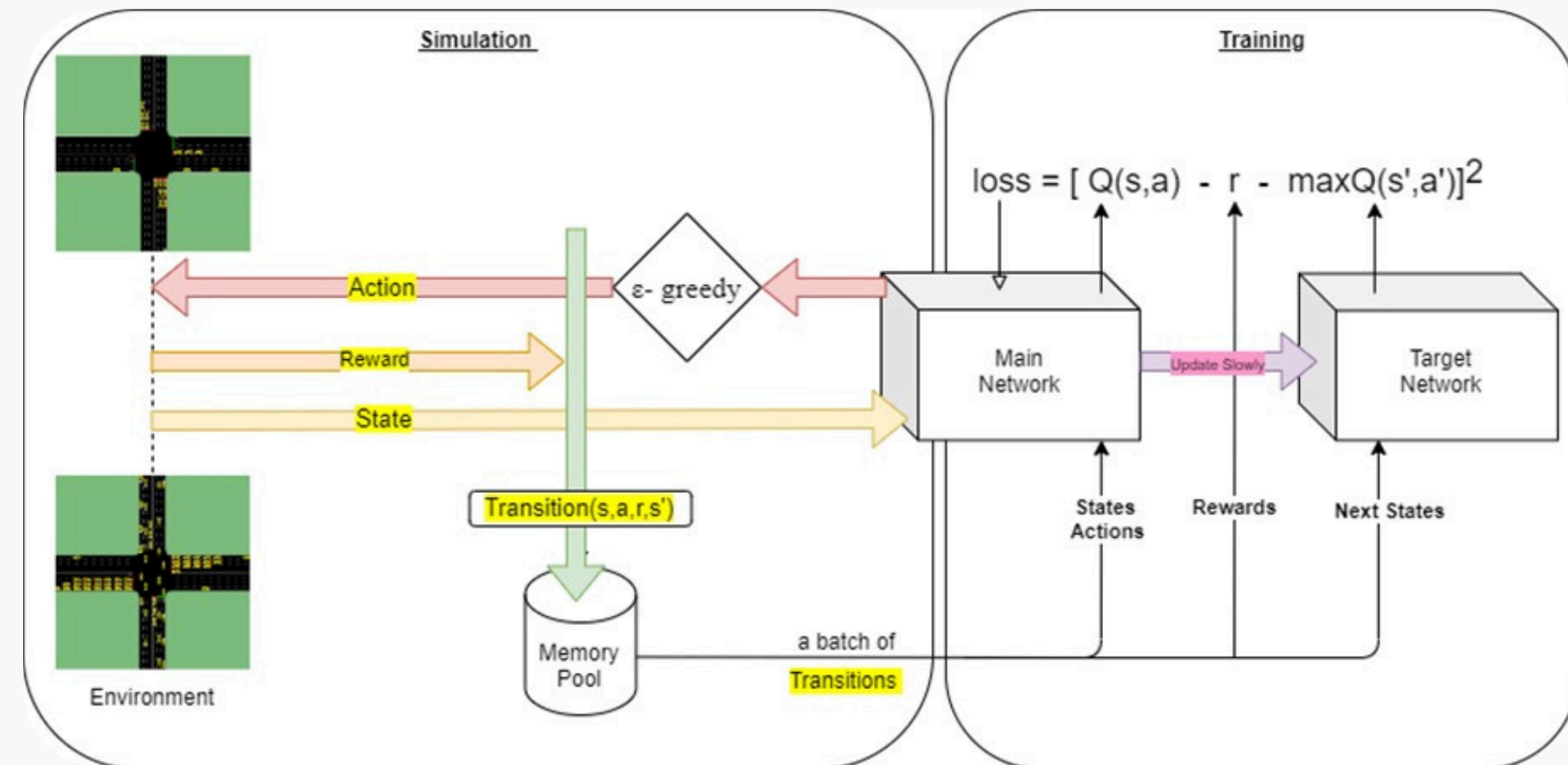
State, Action & Reward Design

State (7 values)

- Vehicle counts from 6 lane detectors
- Current traffic signal phase

Actions

- $0 \Rightarrow$ keep phase unchanged
- $1 \Rightarrow$ switch to next phase
- $R = -(\text{sum of queue lengths})$
- A lower queue $\rightarrow$ higher reward $\rightarrow$ better policy.



Static Controller

Key Characteristics:

- Works on fixed signal durations with no change during the simulation.
- Does not learn or adapt based on traffic flow.
- Repeats the same timing pattern, even when traffic increases or decreases

Limitation:

- Performs poorly when traffic becomes uneven or unpredictable.
- Causes long queues and higher delays because timings don't adjust to demand.

Q-Learning Controller

What is Q-Learning?

A tabular, model-free RL algorithm that learns the value of taking an action in a given state using trial-and-error.

Working

- Observe state (s): queue lengths + signal phase
- Take action (a): keep or switch phase
- Receive reward (R): -(total queue)
- Update Q-table:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Deep Q-Network (DQN) Controller

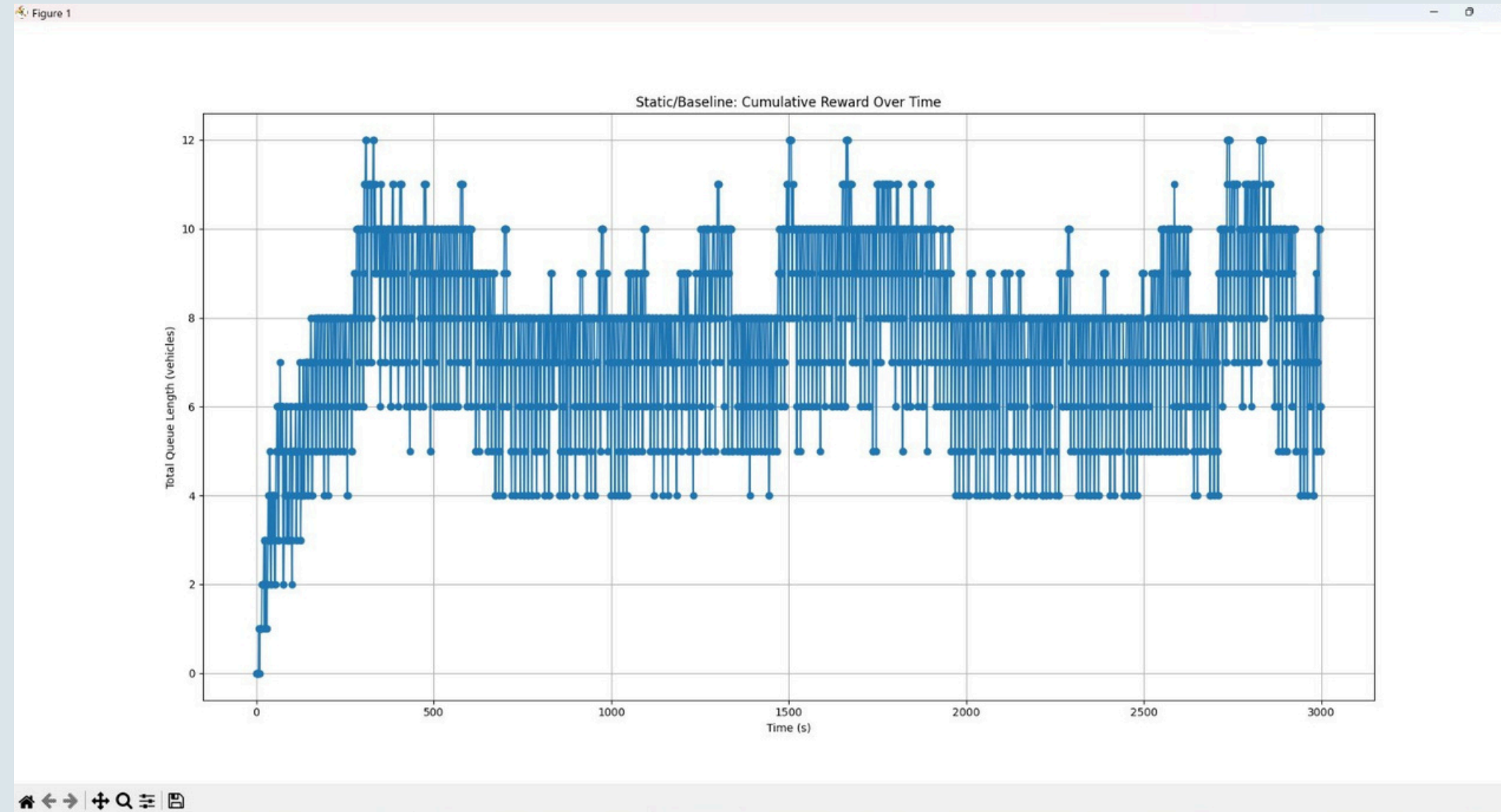
What is DQN?

A neural network-based RL algorithm that approximates Q-values instead of storing a Q-table, allowing it to handle larger and continuous state spaces.

Working

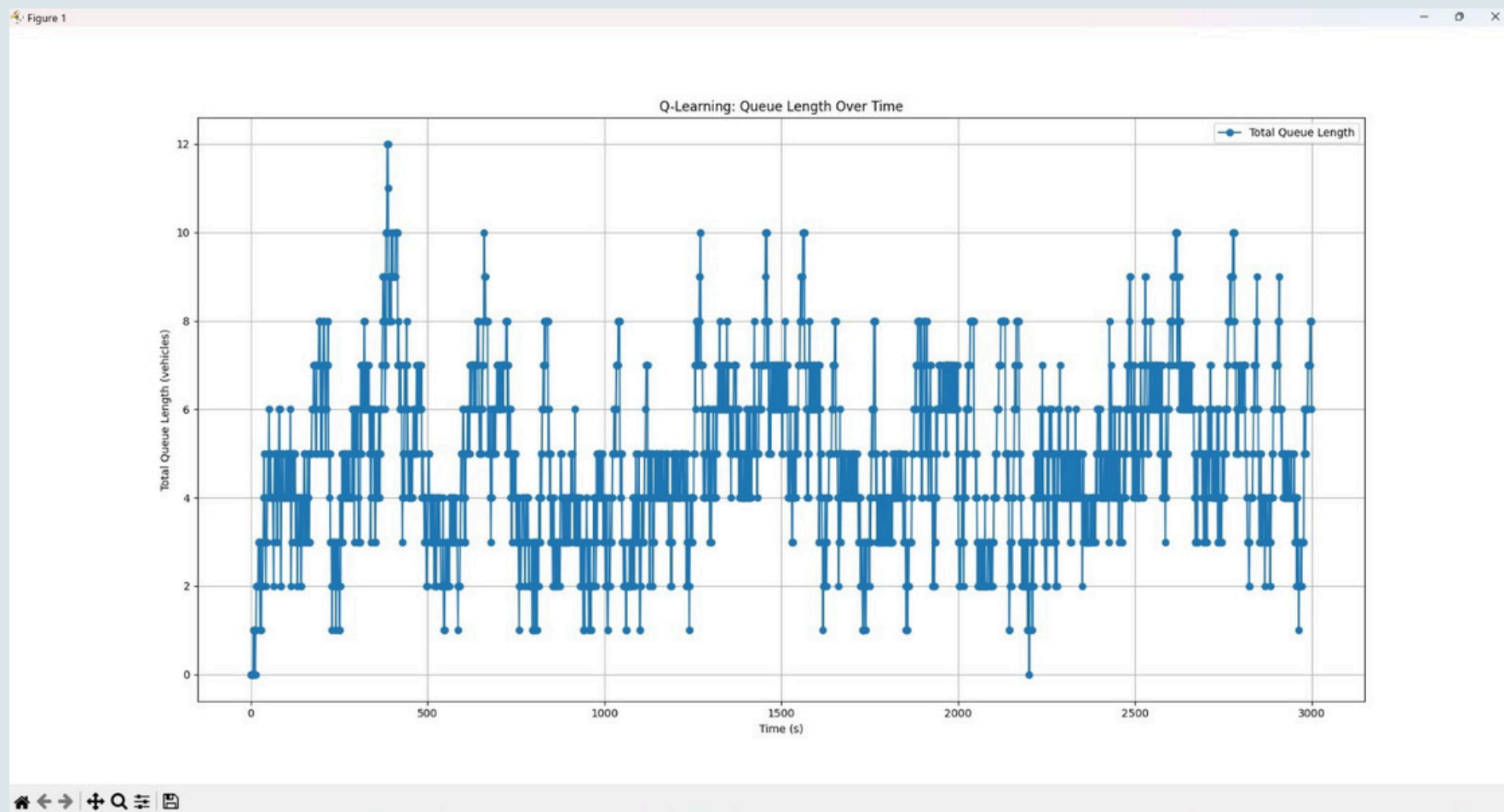
- Neural network replaces Q-table for estimating Q-values
 - Input: 6 detector queues + signal phase
 - Output: Q-values for actions (keep / switch)
 - Chooses the action with highest Q-value
 - Reward = $-(\text{total queue})$
 - Network updated via backpropagation each step
 - Learns stable, adaptive signal switching over time
-

Static Model: Queue Length Graph

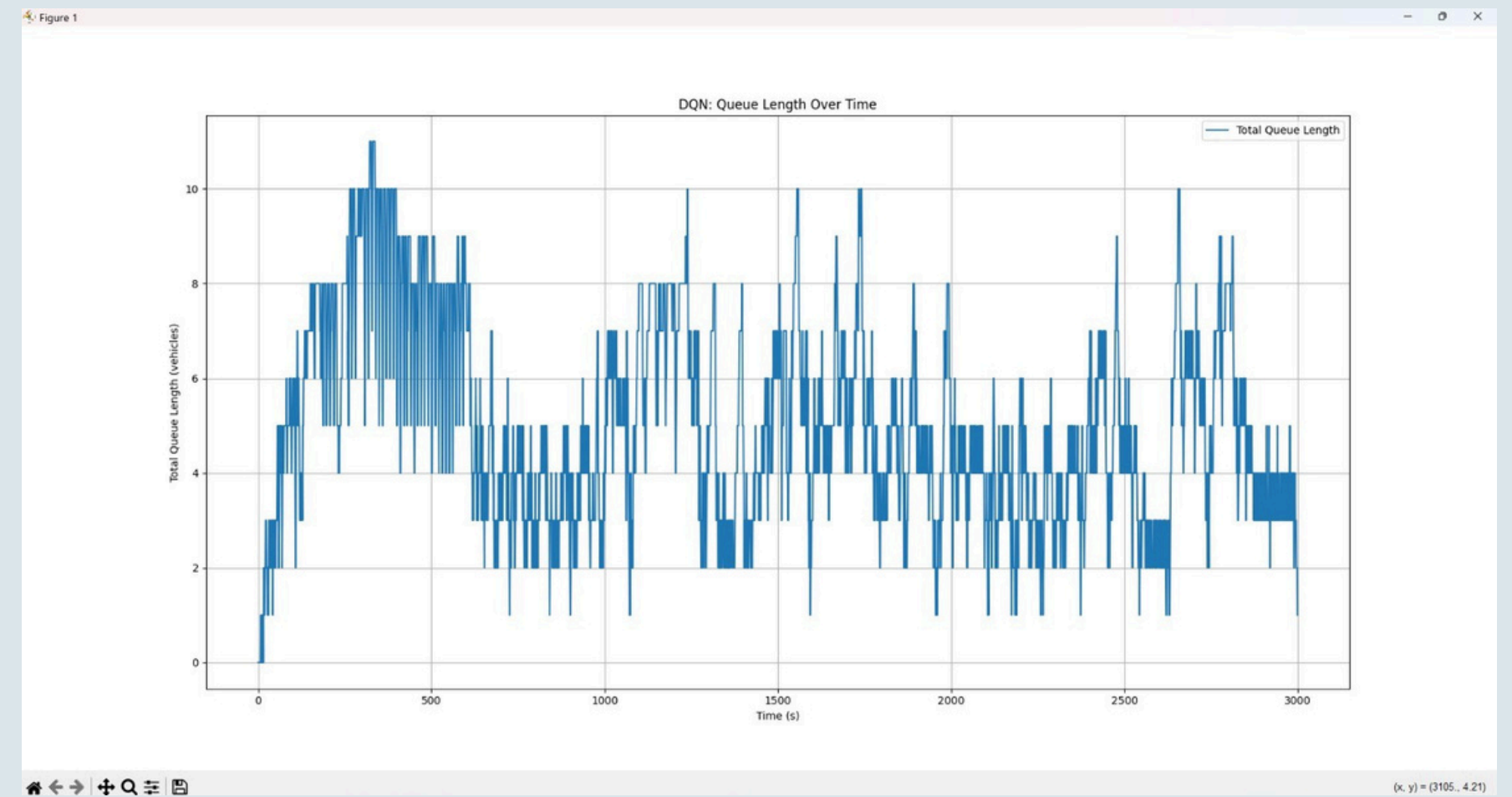


- This graph represents queue length over the full simulation duration under fixed-time traffic control.
- The queue frequently stays between 7–12 vehicles, with many spikes.
- There is no learning or adaptation, so the signal does not respond to changing traffic.

Q-Learning vs DQN: Queue Length Comparison



- Queue values significantly lower than static.
- Queue stays mostly between 3–6 vehicles instead of 7–12.



- Smooth, stable queue behavior.
- Learns generalized patterns → less oscillation.
- Queue consistently between 3–5 vehicles.

Static vs Q model : Python Time Analysis Output

```
=== STATIC METRICS ===  
Total delay (veh-seconds): 23301.00  
Average queue length (vehicles): 7.77  
Max queue length (vehicles): 12  
Vehicles completed (throughput): 1929  
Average travel time per vehicle (s): 287.54  
90th percentile travel time (s): 409.00  
Max travel time (s): 1332.00
```

- Total Delay extremely high (23k vehicle-seconds) → system is slow.
- Average queue: 7.77 vehicles → matches queue graph behavior.
- Max queue: 12 vehicles → static fails to clear congestion.
- Throughput lowest (1929) → static clears fewer vehicles.
- Travel time worst: 1332 seconds maximum → severe congestion.

```
=== Q-LEARNING METRICS ===  
Total delay (veh-seconds): 14344.00  
Average queue length (vehicles): 4.78  
Max queue length (vehicles): 12  
Vehicles completed (throughput): 2368  
Average travel time per vehicle (s): 123.61  
90th percentile travel time (s): 172.00  
Max travel time (s): 1325.00
```

- Delay reduced by ~39%
- Avg travel time reduced by ~57%
- Average queue reduced by ~38%
- Highest throughput → 2368 vehicles
- Learns quickly and aggressively

Static vs DQN: Python Output Comparison

```
=== STATIC METRICS ===  
Total delay (veh-seconds): 23301.00  
Average queue length (vehicles): 7.77  
Max queue length (vehicles): 12  
Vehicles completed (throughput): 1929  
Average travel time per vehicle (s): 287.54  
90th percentile travel time (s): 409.00  
Max travel time (s): 1332.00
```

- Total Delay extremely high (23k vehicle-seconds) → system is slow.
- Average queue: 7.77 vehicles → matches queue graph behavior.
- Max queue: 12 vehicles → static fails to clear congestion.
- Throughput lowest (1929) → static clears fewer vehicles.
- Travel time worst: 1332 seconds maximum → severe congestion.

```
=== DQN METRICS ===  
Total delay (veh-seconds): 14966.00  
Average queue length (vehicles): 4.99  
Max queue length (vehicles): 11  
Vehicles completed (throughput): 2303  
Average travel time per vehicle (s): 130.08  
90th percentile travel time (s): 191.00  
Max travel time (s): 973.00
```

- Delay reduced by ~36% compared to static
- Avg travel time reduced by ~54%
- Average queue reduced by ~35%
- Throughput remains high → 2303 vehicles
- Learns smoothly and conservatively, avoiding aggressive switching

Final Comparison Table

Metric	Static	Q-Learning	DQN
Total Delay (veh-s)	23301	14344	14966
Avg Queue	7.77	4.78	4.99
Max Queue	12	12	11
Vehicles Completed	1929	2368	2303
Avg Travel Time	287.54	123.61	130.08
90th Perc TT	409	172	191
Max TT	1332	1325	973

- Q-Learning and DQN both give better performance than Static across all major metrics.
- Q-Learning shows the best improvement in delay, queue length, and travel time.
- DQN is more stable and gives the lowest maximum travel time, handling congestion spikes better.
- The Static controller struggles the most, showing the highest delays and longest queues.

Observations

- RL-based controllers clearly outperform static timing under identical conditions
- RL reduces congestion by 35–40%
- Q-Learning achieves the lowest average delay and highest throughput
- DQN provides more stable switching and best worst-case travel time
- Static controller produces long, persistent queues
- RL agents learn to switch only when needed, reducing unnecessary phase changes
- Queue spikes reduce significantly under both RL models
- DQN handles heavy congestion more effectively due to neural generalization

Future Scope

- Extend to multi-intersection and network-level traffic control
- Implement Multi-Agent RL (MARL) for coordinated signals
- Integrate pedestrian crossings, emergency vehicle priority
- Replace SUMO detectors with real CCTV/YOLO-based detection
- Deploy RL controller on Jetson Nano / Raspberry Pi for real-time demos
- Use real-world datasets (NHAI/CCTV feeds) for training and validation

Conclusion

- Reinforcement Learning enables real-time adaptive traffic signal control
- Q-Learning and DQN significantly outperform static timing
- Delay reduced by 35–40%, queues reduced by 38%+, throughput improved
- Q-Learning excels in average efficiency, while DQN excels in stability
- Demonstrates feasibility of AI-driven ITS for smart cities
- Provides a scalable foundation for real-world deployment across urban intersections

References

1. Wei et al., PressLight: Learning Max-Pressure Control for Urban Traffic, KDD 2019
2. Zheng et al., IntelliLight: RL for Adaptive Traffic Control, KDD 2019
3. Van der Pol & Oliehoek, Coordinated Deep RL for Traffic Light Control, NeurIPS Workshop, 2016
4. Aslani et al., Adaptive Traffic Signal Control Using Deep RL, Expert Systems with Applications, 2022
5. Lin & Zheng, Efficient Deep RL for Signal Control, IEEE Trans. Vehicular Tech., 2022
6. Lopez et al., SUMO Microscopic Traffic Simulator, IEEE ITSC, 2018
7. Sutton & Barto, Reinforcement Learning: An Introduction, MIT Press, 2018